

ADD aplicado a E-commerce

Kata de Attribute Driven Design sobre dominio de e-commerce con pagos externos

Ingeniería de Software II · ITBA · 2026-1C · Ejercicio de clase 3

RESUMEN

Kata de clase 3: aplicar **Attribute Driven Design** a una plataforma de e-commerce multicanal con procesadores de pago externos. Se elige un **top-4 de atributos** (Security · Availability · Scalability · Interoperability), se arma el blueprint con todos los sistemas externos y se itera por escenarios. Aparece de paso la distinción **OLTP vs OLAP**.

§0 El enunciado

Plataforma de e-commerce con múltiples canales y pagos delegados a terceros. Hay que **diseñar la arquitectura aplicando ADD**, no saltar a dibujar cajas (source: ejercicio-clase-add-ecommerce.md).

DEFINICIÓN

ADD (Attribute Driven Design): método iterativo de diseño dirigido por los **atributos de calidad**: se eligen los drivers, se propone una arquitectura candidata, se generan escenarios y se modifica la arquitectura hasta cubrirlos (Bass, Clements, Kazman, SAIP; ver [[Attribute Driven Design]]).

Dominio (inferido del diagrama de pizarra)

- **Canales de entrada:** sitio **E-commerce** (web), **Tablets** en puntos físicos, **Depósito / envío** (back office).
- **Capas internas:** SPA + LB (load balancer) frente a un **Componente** de aplicación central.
- **Pagos externos:** Procesador Pago 1 y Procesador Pago 2 (Prisma, Mercado Pago, etc. — sin control del equipo).
- **Persistencia:** SQL **OLTP** con replicación **Primario / Secundario**.

Las notas a mano del alumno son parciales: la clase se corta en el Paso 2; los pasos siguientes se reconstruyen por método (source: ejercicio-clase-add-ecommerce.md).

§1 Top-4 de atributos (drivers)

SECURITY**AVAILABILITY****SCALABILITY****INTEROPERABILITY**

TABLA 1. Lista candidata y selección en clase

Atributo	¿Driver?	Justificación
Security	Sí	Manejo de pagos y datos de tarjeta — compliance PCI-DSS.
Availability	Sí	Caída $\Rightarrow$ pérdida directa de ventas y reputación.
Scalability	Sí	Picos comerciales: CyberMonday, Hot Sale, Black Friday.
Interoperability	Sí	Múltiples procesadores de pago + sistemas de logística.
Performance	No	"Lo dejamos afuera" — se considera derivado de Scalability.
Customizability	No	"Lo dejamos afuera" — no crítica para el dominio.

REGLA DE ORO

Heurística del top-4: de la lista candidata se eligen hasta 4 drivers. El resto no se ignora: pasa a la lista de **riesgos asumidos**, que se documentan pero no guían la arquitectura (source: clase-03-ejercicio-add-ecommerce.md).

CUÁNDO NO • CUIDADO

No tratar **todos** los atributos como drivers: agrega ruido y vuelve indecible el diseño. Tampoco saltar al dibujo de cajas antes de articular los atributos — es el anti-patrón #1 de la kata.

§2 Los 5 pasos de ADD aplicados

Paso 1 — Elegir los atributos driver

Sobre los requerimientos funcionales se arma la lista candidata y se selecciona el top-4 (ver §1). Performance y Customizability quedan como riesgos asumidos (source: ejercicio-clase-add-ecommerce.md).

Paso 2 — Arquitectura candidata con sistemas externos

La primera versión incluye TODOS los sistemas externos sobre los que el equipo no tiene control (procesadores de pago, depósito/envío, dispositivos cliente) más el mínimo de componentes internos para cumplir los requerimientos funcionales (source: ejercicio-clase-add-ecommerce.md).

Sistema externo (def. operativa de la cátedra): *si no podés cambiarle el código ni el contrato, es externo* — y entra de primera clase al blueprint inicial (source: clase-03-ejercicio-add-ecommerce.md; ver [[Clase 4 — ¿Cuándo diseñamos?]]).

Paso 3 — Generar escenarios por atributo

Un escenario concreto por cada driver, en orden de prioridad (reconstruido por método, source: ejercicio-clase-add-ecommerce.md):

Security

Un atacante intenta inyectar SQL en el checkout → la app sanitiza inputs y el WAF bloquea el patrón antes del backend.

Availability

Un nodo del LB cae → el LB redirige el tráfico al nodo sano sin que el usuario lo perciba.

Scalability

Durante Hot Sale el tráfico crece 50× → autoscaling horizontal absorbe la carga; las DBs Primario+Secundario absorben los reads vía replicación.

Interoperability

Se incorpora un nuevo procesador de pago → adapter pluggable con interfaz común, sin tocar el componente core.

Paso 4 — Evaluar y modificar la arquitectura

Para cada escenario se verifica si la candidata lo sobrevive; si no, se modifica **re-validando los escenarios anteriores** y los requerimientos funcionales (source: ejercicio-clase-add-ecommerce.md).

Paso 5 — Iterar

Repetir hasta que todos los escenarios estén cubiertos o asumidos como riesgo.

§3 Diagrama de componentes

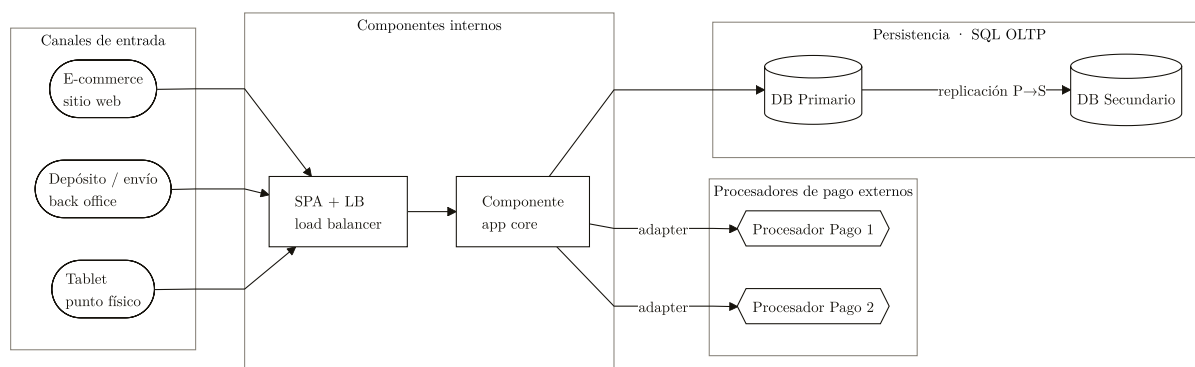


FIGURA 1. *Blueprint ADD: tres canales entran por SPA+LB al componente core, que delega pagos a dos procesadores externos vía adapters y persiste en SQL OLTP con réplica Primario→Secundario.*

SPA + LB + Componente + DB es un **4-tier** clásico de **Hierarchical Layers** (ver [\[\[Hierarchical Layers\]\]](#)). Los pagos son sistemas externos colgados del core vía adapters.

§4 Trade-offs

TABLA 2. Tensiones de la arquitectura

Decisión	Gana	Resigna
WAF + sanitización en checkout	Security	Performance / latencia
Réplica Primario/Secundario	Availability + reads escalables	Consistency (lag de réplica)
Adapter pluggable por procesador	Interoperability	Complejidad / indirección

TRADE-OFF

Availability ↔ Consistency: replicar a un secundario da disponibilidad y absorbe lecturas, pero introduce lag de réplica: un read al secundario puede ver datos viejos. Se resuelve enviando lecturas críticas (saldo, stock en checkout) al primario y el reporting al secundario.

TRADE-OFF

Security ↔ Performance: Performance se descartó como driver, pero el WAF y la sanitización agregan latencia real. Como Performance es riesgo asumido, se acepta el costo siempre que no degrade los SLAs de Scalability.

§5 Tema lateral: OLTP vs OLAP (primer encuentro)

Surge al preguntarse "¿dónde guardo las métricas de uso (scroll-depth, precio del dólar al momento de la venta)?" (source: ejercicio-clase-add-ecommerce.md).

TABLA 3. OLTP vs OLAP

Tipo	Real-time	Forma	Uso típico
OLTP — Online Transaction Processing	Sí	Esquema normalizado, transaccional (ACID)	Carrito, checkout, inventario
OLAP — Online Analytical Processing	No	Desnormalizado, append-only , "tabla gigante"	Métricas, analytics, snapshots

FUENTE

"Muy bueno para métricas, append-only en una tabla gigante con un montón de datos. Ej: cantidad de píxeles scrollados, precio del dólar en el momento de la venta, etc." — sobre OLAP (source: ejercicio-clase-add-ecommerce.md).

CUÁNDO SÍ • VENTAJAS

Criterio cátedra: separar OLAP de OLTP. El OLTP no debe servir reporting pesado — patrones de carga distintos. Profundizado en [[Clase 6 — Persistencia]] y [[OLAP y ETL]] (source: clase-03-ejercicio-add-ecommerce.md).

EN EL PARCIAL

Respuesta mínima de esta kata: top-4 con justificación + blueprint con sistemas externos + escenario por atributo + 2 trade-offs explícitos. Errores que matan la respuesta: dibujar cajas sin articular atributos; tratar todos los atributos como drivers; meter las métricas en el OLTP.

EJEMPLO

Variante: si el negocio pivotea de B2C minorista a **marketplace** (vendedores terceros), entran *Trust* y *Customizability* — ¿desplazan a Scalability del top-4? (source: clase-03-ejercicio-add-ecommerce.md).